Parte da Aula 1

Flutter + API Java – App de Empréstimo de EPIs

Apresentação da aula

Nesta aula a turma constrói um app Flutter em que o funcionário faz login, consulta o catálogo de EPIs e registra o empréstimo e a devolução de equipamentos, tudo ligado a uma API Java (Spring Boot) com banco MySQL. O cadastro de EPIs e de funcionários pertence a outro sistema da empresa; aqui esses dados já chegam prontos no banco, e o app só consulta, faz o login e movimenta os empréstimos.

Por que esse tema. A NR-06 obriga a empresa a fornecer o EPI adequado, com Certificado de Aprovação (CA) válido, e a registrar a entrega ao trabalhador. Controlar quem está com qual equipamento e se o CA está vencido é um problema real do técnico de Segurança do Trabalho, e é a base do projeto integrador da turma.

Objetivos de aprendizagem

Ao final, o aluno será capaz de:

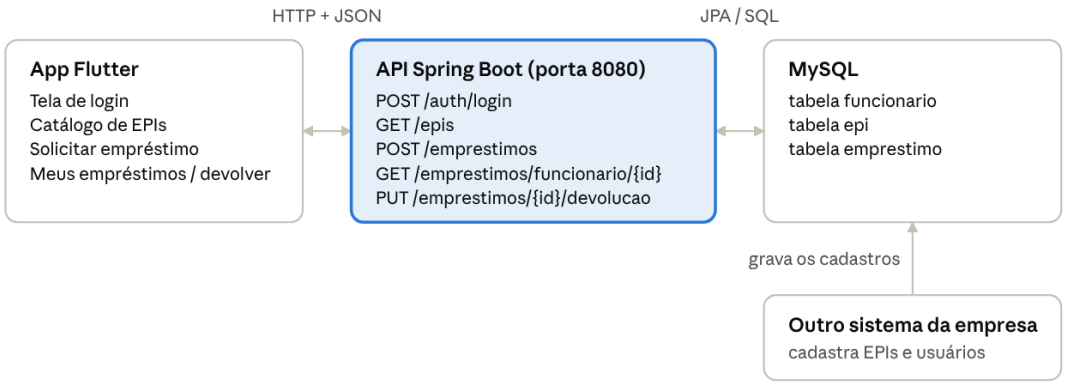
1. Mapear entidades JPA com relacionamento `@ManyToOne` (empréstimo → funcionário e EPI).
2. Criar endpoints REST com GET, POST e PUT e aplicar regras de negócio na API.
3. Implementar um login simples por matrícula e senha.
4. Consumir a API no Flutter com o pacote `http`, convertendo JSON em objetos Dart.
5. Navegar entre telas passando o funcionário logado e atualizar a tela após uma ação.

Pré-requisitos

- Projeto Spring Boot funcionando (dependências Spring Web, Spring Data JPA e MySQL Driver).
- MySQL rodando, com o banco já configurado no `application.properties`.
- Projeto Flutter com o pacote `http` instalado.
- Postman instalado.

A arquitetura completa aparece no diagrama abaixo: o outro sistema grava os cadastros no MySQL, a API lê e movimenta esses dados, e o app Flutter conversa somente com a API.

O app só fala com a API; os cadastros chegam de outro sistema



arquitetura do app de empréstimo de EPIs · 3 camadas + sistema de cadastro

Etapa 1 – SpringBoot, banco e Configurações

Estágio 1 – Criação da aplicação Springboot

Abrir o Spring Initializr - https://start.spring.io

Configure exatamente assim:

Project Maven

Language Java

Spring Boot Escolha a versão estável mais recente.(4.1.2)

Group com.senac

Artifact api_epi

Name api_epi

Package com.senac.api_epi

Packaging Jar

Java 17 (ver antes a versão do laboratório)

Adicionar dependências - Add Dependencies

Adicione: Spring Web, Spring Data JPA, **Banco:** MySQL Driver, **Desenvolvimento:** Lombok (Opcional, mas ajuda muito)

Generate - Clique: Generate

Vai baixar: api_epi.zip

Extraia o ZIP na pasta onde ficam suas aplicações

Explicação das dependências:

1. Spring Web

Função: Criar a API REST e receber/enviar requisições HTTP.

É a dependência que permite criar endpoints como:

GET /funcionarios

POST /funcionarios

PUT /funcionarios/1

DELETE /funcionarios/1

Principais anotações:

@RestController @RequestMapping @GetMapping @PostMapping @PutMapping @DeleteMapping

Analogia: é a "porta de entrada" da aplicação.

2. Spring Data JPA

Função: Facilitar o acesso ao banco de dados.

Sem ela, você precisaria escrever muitas consultas SQL e código de conexão manualmente.

Com JPA:

public interface FuncionarioRepository

extends JpaRepository<Funcionario, Long> {

}

Você ganha métodos prontos:

findAll() / findById() / save() / deleteById()

Analogia: é o "tradutor" entre objetos Java e tabelas do banco.

3. MySQL Driver

Função: Permitir que o Java converse com o MySQL.

É o conector JDBC. Sem essa dependência, o Spring não consegue abrir conexão com o banco.

Quando você escreve:

.properties

spring.datasource.url=jdbc:mysql://localhost:3306/epi_db

quem faz a comunicação é o Driver MySQL.

Analogia: é a "ponte" entre a aplicação e o banco.

4. Lombok

Função: Reduzir código repetitivo.

Sem Lombok:

Java

public class Funcionario {

private Long id;
private String nome;

public Long getId() {
return id;
}

```
public void setId(Long id) {
this.id = id;
}

public String getNome() {
return nome;
}

public void setNome(String nome) {
this.nome = nome;
}
}
```

Com Lombok:

```
@Data
public class Funcionario {

private Long id;
private String nome;

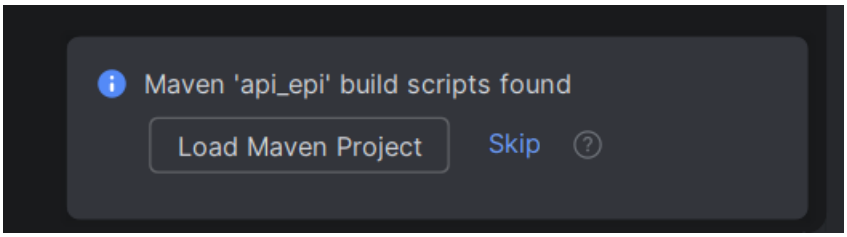
}
```

O Lombok gera automaticamente: Getters Setters toString() equals() hashCode()

Analogia: é um "assistente" que escreve código repetitivo para você.

Estágio 2 – Abra a IDE e carrega o projeto Maven da pasta extraída

Se você estiver usando o InteliJ pod aparecer essa mensagem



Nessa janela, clique em: **Load Maven Project - Por quê?**
O arquivo que você baixou do Spring Initializr é um projeto **Maven**. Dentro dele existe um arquivo chamado: pom.xml
Esse arquivo informa ao IntelliJ:

- qual versão do Java usar;
- quais dependências baixar (Spring Web, JPA, MySQL Driver, Lombok);
- como compilar o projeto;
- como executar a aplicação.

Após aguarde o download no canto direito inferior

Estágio 3 – Crie no servidor do mysql o banco: epi_db

Estágio 4 – Configure o banco no seu projeto Mavem (springboot)

- src/main/resources/application.properties
- configuração:

```
spring.datasource.url=jdbc:mysql://localhost:3306/epi_db
/*
spring.datasource.url=jdbc:mysql://localhost:3306/epi_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=America/Sao_Paulo
*/

spring.datasource.username=root
spring.datasource.password=SUA_SENHA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=8080
```

Estágio 5 – Teste a aplicação

Rode a aplicação para ver se houve algum erro no console

Etapa 2 – Tabelas / Entidades

Nesta parte criaremos três tabelas do sistema (funcionário, EPI e empréstimo) carregando os dados que, na empresa real, viriam do sistema de cadastro. Use o mesmo projeto Spring Boot da app, com os pacotes `entities`, `repositories` e `controllers`.

Crie os pacotes abaixo, seguindo a hierarquia:

```
com.senac.api_epi
 │
 ├── controllers
 ├── entities
 ├── repositories
 ├── dto
 ├── service
 └── config
```

1. Criar a entidade Funcionario

Na pasta `com.senac.api_epi.entities`, crie a classe `Funcionario`. A anotação `@JsonProperty(access = WRITE_ONLY)` faz a API receber a senha, mas nunca devolvê-la no JSON.

```
package com.senac.api_epi.entities;

import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.persistence.*;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
public class Funcionario {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @Column(nullable = false, unique = true, length = 20)
   private String matricula;
```

```
    @Column(nullable = false, length = 120)
    private String nome;

    @Column(nullable = false, length = 80)
    private String setor;

    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    @Column(nullable = false, length = 60)
    private String senha;
}
```

Obs: estão faltando os Getters e Settres?

2. Criar a entidade Epi (classe)

```
      package com.senac.api_epi.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

import java.time.LocalDate;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
public class Epi {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String nome;

    @Column(nullable = false, length = 80)
    private String categoria;

    @Column(nullable = false, length = 20)
    private String ca;

    @Column(nullable = false)
    private LocalDate validadeCa;

    @Column(nullable = false)
```

```
    private String imagemUrl;

    @Column(nullable = false)
    private int quantidadeDisponivel;

    public boolean isCaVencido() {
        return validadeCa.isBefore(LocalDate.now());
    }
}
```

3. Criar a entidade Emprestimo (Classe)

O empréstimo liga um funcionário a um EPI. A anotação `@ManyToOne` significa: muitos empréstimos para um mesmo funcionário, e muitos empréstimos para um mesmo EPI.

```
package com.senac.api_epi.entities;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDate;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
public class Emprestimo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false)
    @JoinColumn(name = "funcionario_id")
    private Funcionario funcionario;

    @ManyToOne(optional = false)
    @JoinColumn(name = "epi_id")
    private Epi epi;

    @Column(nullable = false)
    private LocalDate dataEmprestimo;

    private LocalDate dataDevolucao;

    @Column(nullable = false, length = 15)
    private String status;
}
```

4. Criar os repositórios

Na pasta `com.senac.api_epi.repositories`, **crie três interfaces**. O Spring monta a consulta SQL a partir do nome do método: `findByMatricula` vira `WHERE matricula = ?`.

Funcionariorepository:

```
package com.senac.api_epi.repositories;

import com.senac.api_epi.entities.Funcionario;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface FuncionarioRepository extends JpaRepository<Funcionario, Long> {

    Optional<Funcionario> findByMatricula(String matricula);

}
```

EpiRepository:

```
package com.senac.api_epi.repositories;

import com.senac.api_epi.entities.Epi;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EpiRepository extends JpaRepository<Epi, Long> {

}
```

EmprestimoRepository:

```
package com.senac.api_epi.repositories;

import com.senac.api_epi.entities.Emprestimo;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface EmprestimoRepository extends JpaRepository<Emprestimo, Long> {

    // Empréstimos de um funcionário, do mais recente para o mais antigo
    List<Emprestimo> findByFuncionarioIdOrderByIdDesc(Long funcionarioId);

    // O funcionário já está com este EPI e ainda não devolveu?
```

```
    boolean existsByFuncionarioIdAndEpiIdAndStatus(
            Long funcionarioId,
            Long epiId,
            String status
    );

}
```

5. Configurar CORS e a pasta de imagens (WebConfig)

`Crie a classe WebConfig`.

Se ainda houver `@CrossOrigin(origins = "http://localhost:8000")` nos controllers antigos, apague: o CORS fica só aqui.

O que é CORS?
Flutter Web http://localhost:8000
tentando acessar
Spring Boot http://localhost:8080
Como são portas diferentes, o navegador entende que são origens diferentes.
Sem CORS ativado você recebe erros como:
Access to fetch has been blocked by CORS policy
Essa configuração libera o acesso.

```
       package com.senac.api_epi.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    // Permite que aplicações externas consumam a API
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOriginPatterns("*")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true);
    }

    // Disponibiliza imagens da pasta static/imagens
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/imagens/**")
                .addResourceLocations("classpath:/static/imagens/");
    }
}
```

Pasta das imagens

Crie a estrutura:
Plain Text
src
└── main
    └── resources
        └── static
            └── imagens

No IntelliJ:
resources
→ New
→ Directory
Crie:
static
Depois:
imagens

6. Arquivos sugeridos
Dentro de: static/imagens
coloque:
capacete.jpg
plug.jpg
concha.jpg
pff2.jpg
luva.jpg
oculos.jpg
cinto.jpg
botina.jpg

6. Rodar a API e conferir as tabelas

Rode o projeto. O Hibernate cria as tabelas `funcionario`, `epi` e `emprestimo`. Confira no MySQL Workbench. Repare que o nome dos campos em Java (camelCase) vira coluna com sublinhado no banco:

Campo no Java	Coluna no MySQL
`validadeCa`	`validade_ca`
`imagemUrl`	`imagem_url`
`quantidadeDisponivel`	`quantidade_disponivel`
`dataEmprestimo`	`data_emprestimo`

7. Carregar os dados do "outro sistema"

Na empresa, quem cadastra EPIs e funcionários é outro sistema. Para simular isso, rode o script abaixo no MySQL Workbench depois que as tabelas existirem. Os números de CA são fictícios, criados para a aula. Dois EPIs são de propósito problemáticos: os óculos têm CA vencido e o protetor tipo concha está sem estoque.

```
      USE epi_db;

INSERT INTO funcionario (matricula, nome, setor, senha) VALUES
('1001', 'Ana Souza', 'Manutenção', '1234'),
('1002', 'Bruno Lima', 'Almoxarifado', '1234'),
('1003', 'Carla Mendes', 'Produção', '1234');

INSERT INTO epi (nome, categoria, ca, validade_ca, imagem_url, quantidade_disponivel) VALUES
('Capacete de segurança classe B', 'Proteção da Cabeça', '40101', '2028-03-15',
'/imagens/capacete.jpg', 10),
('Protetor auricular tipo plug', 'Proteção Auditiva', '40102', '2027-11-30', '/imagens/plug.jpg',
25),
('Protetor auricular tipo concha', 'Proteção Auditiva', '40103', '2027-08-20',
'/imagens/concha.jpg', 0),
('Máscara PFF2', 'Proteção Respiratória', '40104', '2028-01-10', '/imagens/pff2.jpg', 30),
('Luva nitrílica', 'Proteção das Mãos', '40105', '2027-05-05', '/imagens/luva.jpg', 20),
('Óculos de proteção incolor', 'Proteção dos Olhos e Face', '40106', '2025-06-30',
'/imagens/oculos.jpg', 12),
('Cinto de segurança tipo paraquedista', 'Proteção Contra Quedas', '40107', '2028-09-01',
'/imagens/cinto.jpg', 4),
('Botina de segurança', 'Proteção dos Pés', '40108', '2027-12-12', '/imagens/botina.jpg', 8);
```

O campo `imagem_url` guarda só o caminho (`/imagens/capacete.jpg`). O app junta esse caminho ao endereço do servidor. Assim, trocar `localhost` pelo IP da máquina ou por `10.0.2.2` (emulador Android) não exige mexer no banco.

> *Atenção: aqui a senha está gravada como texto puro só para simplificar a aula. Em sistema real, a senha é guardada com hash, por exemplo com BCrypt. Esse é o desafio extra no fim da aula.*

Teste no navegador: `http://localhost:8080/imagens/capacete.jpg` deve abrir a foto.

Obs: Nesse momento você já tem:

- Banco criado
- Dados inseridos
- Entidades (Funcionario, Epi, Emprestimo)
- Repositories
- Spring Boot funcionando
- Conexão com MySQL funcionando

O que falta é apenas um endpoint temporário para expor os dados.

8. Criar os Endpoint:

No pacote controllers, crie a classe EpiController:

package com.senac.api_epi.controllers;

```
import com.senac.api_epi.entities.Epi;
import com.senac.api_epi.repositories.EpiRepository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class EpiController {

   private final EpiRepository epiRepository;

   public EpiController(EpiRepository epiRepository) {
      this.epiRepository = epiRepository;
   }

   @GetMapping("/epis")
   public List<Epi> listar() {
      return epiRepository.findAll();
   }
}
```

⇨ Testando: Execute novamente a aplicação.
⇨ Agora acesse:
⇨ http://localhost:8080/epis

No pacote controllers, crie a classe FuncionarioController:

```
package com.senac.api_epi.controllers;

import com.senac.api_epi.entities.Funcionario;
import com.senac.api_epi.repositories.FuncionarioRepository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class FuncionarioController {

   private final FuncionarioRepository repository;

   public FuncionarioController(FuncionarioRepository repository) {
      this.repository = repository;
   }
```

```
    @GetMapping("/funcionarios")
    public List<Funcionario> listar() {
        return repository.findAll();
    }
}
```

⇨ Testando: Execute novamente a aplicação.

⇨ Agora acesse: http://localhost:8080/funcionarios

Parte da Aula 2

Etapa 3 – Inciando o projeto Flutter

1 - Criar o projeto Flutter

No terminal: flutter create app_epi

Depois: cd app_epi

Instale o pacote HTTP: flutter pub add http

2 - Criar apenas estas pastas

Dentro de lib:

lib

└── src

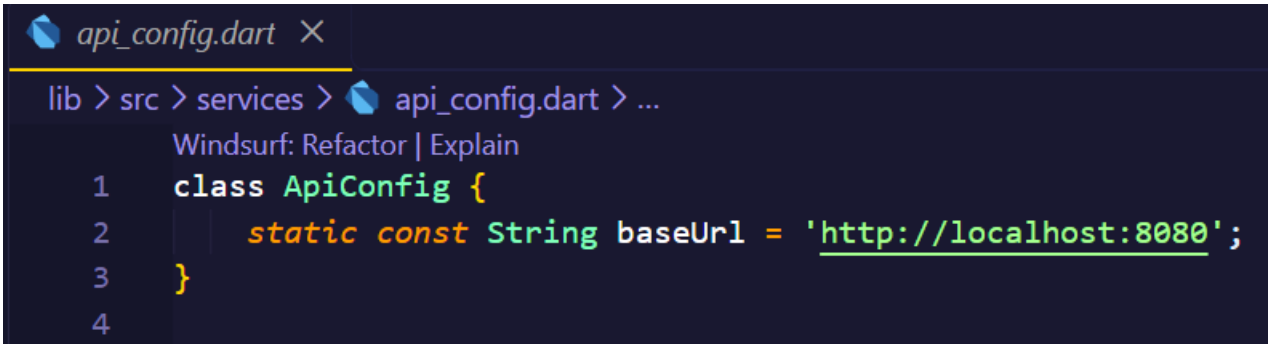
    ├── models

    ├── services

    └── screens

3 - Criar ApiConfig

Arquivo: lib/src/services/api_config.dart



Responda: Olhando a estrutura deste arquivo, qual é o objetivo dele?

4 - Criar model Epi

Arquivo: lib/src/models/epi.dart

epi.dart

lib > src > services > epi.dart > ...

Windsurf: Refactor | Explain

```dart
class Epi {
  final int id;
  final String nome;
  final String categoria;
  final String ca;
  final String imagemUrl;

  Epi({
    required this.id,
    required this.nome,
    required this.categoria,
    required this.ca,
    required this.imagemUrl,
  });

  // Construtor de fábrica que converte um JSON recebido da API
  // em um objeto Epi do Flutter.
  // Cada campo do JSON é associado ao atributo correspondente da classe.
  factory Epi.fromJson(Map<String, dynamic> json) {
    return Epi(
      id: json['id'],
      nome: json['nome'],
      categoria: json['categoria'],
      ca: json['ca'],
      imagemUrl: json['imagemUrl'],
    );
  }
}

```

5 - Criar service

Arquivo: lib/src/services/epi_service.dart

epi_service.dart

lib > src > services > epi_service.dart > EpiService

```dart
import 'dart:convert'; // Trabalha com JSON
import 'package:http/http.dart' as http; // Faz requisições HTTP

import '../models/epi.dart';
import 'api_config.dart';

Windsurf: Refactor | Explain
class EpiService {

  // Busca a lista de EPIs na API
  Windsurf: Refactor | Explain | Generate Function Comment | ×
  Future<List<Epi>> buscarEpis() async {

    // Faz uma requisição GET para o endpoint /epis
    final response =
        await http.get(Uri.parse('${ApiConfig.baseUrl}/epis'));

    // Verifica se a requisição foi bem-sucedida
    if (response.statusCode == 200) {

      // Converte o JSON recebido em uma lista Dart
      List<dynamic> lista =
          json.decode(utf8.decode(response.bodyBytes));

      // Transforma cada item da lista em um objeto Epi
      return lista
          .map((item) => Epi.fromJson(item))
          .toList();

    } else {

      // Lança uma exceção em caso de erro
      throw Exception('Erro ao carregar EPIs');

    }
  }
}
```

6 - Criar tela

Arquivo: lib/src/screens/epi_screen.dart

epi_screen.dart

lib > src > screens > epi_screen.dart > ...

```dart
import 'package:flutter/material.dart';
import '../models/epi.dart';
import '../services/epi_service.dart';

// criar uma StatelessWidget chamada EpiScreen que exibe a lista de EPIs
Windsurf: Refactor | Explain
class EpiScreen extends StatelessWidget {
  const EpiScreen({super.key});

  @override
  Windsurf: Refactor | Explain | Generate Function Comment | ×
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('EPIs')),
      body: FutureBuilder<List<Epi>>(
        // Chama o método que busca os dados da API
        future: EpiService().buscarEpis(),
        builder: (context, snapshot) {
          // Exibe indicador de carregamento enquanto aguarda resposta
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          // Exibe mensagem caso ocorra algum erro
          if (snapshot.hasError) {
            return Center(child: Text(snapshot.error.toString()));
          }
          // Recebe a lista retornada pela API
          final epis = snapshot.data ?? [];
          return ListView.builder(
            // Quantidade de itens da lista
            itemCount: epis.length,
            itemBuilder: (context, index) {
              final epi = epis[index];
              return ListTile(
                // Nome do EPI
                title: Text(epi.nome),
                // Informações complementares
                subtitle: Text('${epi.categoria}\nCA: ${epi.ca}'),
              ); // ListTile
            },
          ); // ListView.builder
        },
      ), // FutureBuilder
    ); // Scaffold
  }
}
```

7 - Ajustar o main.dart

main.dart

lib > main.dart > ...

```dart
import 'package:flutter/material.dart';
import 'src/screens/epi_screen.dart';

Run | Debug | Profile | Windsurf: Refactor | Explain | Generate Funct
void main() {
  runApp(const MyApp());
}

Windsurf: Refactor | Explain
class MyApp extends StatelessWidget {

  const MyApp({super.key});

  @override
  Windsurf: Refactor | Explain | Generate Function Comment | ×
  Widget build(BuildContext context) {

    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: const EpiScreen(),
    ); // MaterialApp
  }
}
```

8 - Executar

Primeiro deixe a API Java e o servidor Mysql rodando
Teste no navegador: http://localhost:8080/epis
Se aparecer JSON, a API está pronta.
Depois rode: flutter run -d chrome
Ou flutter run
Ou rode pelo vsCode

Resultado esperado:

Java funcionando
MySQL funcionando
API funcionando
Flutter consumindo a API

9 – Melhorando a visão

a. Criar a tela principal (menu): Arquivo: lib/src/screens/main_screen.dart

main_screen.dart

lib > src > screens > main_screen.dart > MainScreen > build

```dart
import 'package:flutter/material.dart';

import 'epi_screen.dart';

Windsurf: Refactor | Explain
class MainScreen extends StatelessWidget {
  const MainScreen({super.key});

  @override
  Windsurf: Refactor | Explain | Generate Function Comment | ×
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Empréstimos de EPIs'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ), // AppBar

      drawer: Drawer(
        child: ListView(
          children: [
            const DrawerHeader(
              decoration: BoxDecoration(
                color: Colors.blue,
              ), // BoxDecoration
              child: Text(
                'Menu Principal',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 22,
                ), // TextStyle
              ), // Text
            ), // DrawerHeader
```

```
            ListTile(
              leading: const Icon(Icons.home),
              title: const Text('Início'),
              onTap: () {
                Navigator.pop(context);
              },
            ), // ListTile

            ListTile(
              leading: const Icon(Icons.health_and_safety),
              title: const Text('EPIs'),
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (_) => const EpiScreen(),
                  ), // MaterialPageRoute
                );
              },
            ), // ListTile
          ],
        ), // ListView
      ), // Drawer

      body: const Center(
        child: Text(
          'Selecione uma opção no menu lateral',
          style: TextStyle(fontSize: 18),
        ), // Text
      ), // Center
    ); // Scaffold
  }
}
```

b) alterar o main.dart

veja no seu arquivo somente onde se fará as alterações



c) melhorar a EpiScreen - Veja onde se dá os ajustes

epi_screen.dart

lib > src > screens > epi_screen.dart > EpiScreen > build

```dart
import 'package:flutter/material.dart';
import '../models/epi.dart';
import '../services/epi_service.dart';

Windsurf: Refactor | Explain
class EpiScreen extends StatelessWidget {
  const EpiScreen({super.key});

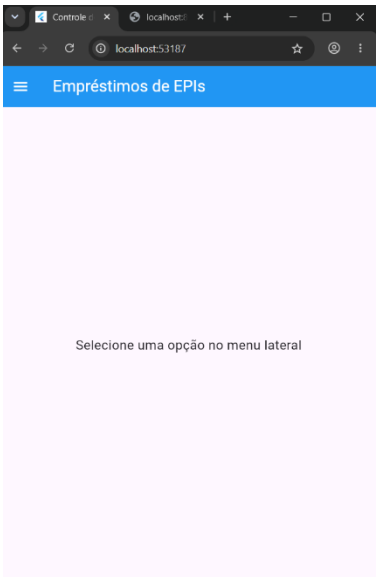
  @override
  Windsurf: Refactor | Explain | Generate Function Comment |
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Lista de EPIs'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,

        actions: [
          IconButton(
            icon: const Icon(Icons.close),
            tooltip: 'Fechar',
            onPressed: () {
              Navigator.pop(context);
            },
          ), // IconButton
        ],
      ), // AppBar
```

```
26
27        body: FutureBuilder<List<Epi>>(
28          // Busca os dados da API
29          future: EpiService().buscarEpis(),
30          builder: (context, snapshot) {
31            // Exibe indicador enquanto aguarda os dados
32            if (snapshot.connectionState ==
33                ConnectionState.waiting) {
34              return const Center(
35                child: CircularProgressIndicator(),
36              ); // Center
37            }
38            // Exibe erro caso ocorra
39            if (snapshot.hasError) {
40              return Center(
41                child: Text(
42                  'Erro: ${snapshot.error}',
43                ), // Text
44              ); // Center
45            }
46            // Recebe a lista retornada pela API
47            final epis = snapshot.data ?? [];
48
49            return ListView.builder(
50              itemCount: epis.length,
51              itemBuilder: (context, index) {
52                final epi = epis[index];
53                return Card(
54                  elevation: 3,
55                  margin: const EdgeInsets.symmetric(
56                    horizontal: 10,
57                    vertical: 5,
58                  ), // EdgeInsets.symmetric
59
```

```
60                  child: ListTile(
61                    leading: const CircleAvatar(
62                      backgroundColor: Colors.blue,
63                      child: Icon(
64                        Icons.health_and_safety,
65                        color: Colors.white,
66                      ), // Icon
67                    ), // CircleAvatar
68
69                    title: Text(epi.nome),
70
71                    subtitle: Text(
72                      '${epi.categoria}\nCA: ${epi.ca}',
73                    ), // Text
74                  ), // ListTile
75                ); // Card
76              },
77            ); // ListView.builder
78          },
79        ), // FutureBuilder
80      ); // Scaffold
81    }
82  }
```

d) rode a aplicação para testar



10 – inserindo a imagem dos EPIs no lugar dos ícones

a) Crie a variável para receber imagem dentro de itemBuilder:

```
return ListView.builder(
  itemCount: epis.length,
  itemBuilder: (context, index) {
    final epi = epis[index];
    final urlImagem = 'http://localhost:8080${epi.imagemUrl}'; // adicionar variavel de ambiente
```

b) Troque essa parte abaixo do seu código:

```
53                  leading: const CircleAvatar(
54                    backgroundColor: Colors.blue,
55                    child: Icon(Icons.health_and_safety, color: Colors.white),
56                  ), // CircleAvatar
57
```

Por esta:

```
leading: ClipRRect(
  borderRadius: BorderRadius.circular(8),

  child: Image.network(
    urlImagem,
    width: 55,
    height: 55,
    fit: BoxFit.cover,

    errorBuilder: (context, error, stackTrace) {
      return Container(
        width: 55,
        height: 55,
        color: Colors.blue.shade50,

        child: const Icon(
          Icons.health_and_safety,
          color: Colors.blue,
          size: 30,
        ), // Icon
      ); // Container
    },
  ), // Image.network
), // ClipRRect
```

Rode a aplicação, caso a imagem não seja encontrado a imagem será trocada por um icone

Esse direcionamento está na linha errorBuilder (indica que se houver erro no carregamento da imagem)

O fluxo da aplicação está nesta ordem:

API Java

↓ JSON

EpiService

↓

Epi Model

↓

EpiScreen

↓

Interface

Exercício

1) Crie a tela para listar todos os funcionários e insira esta tela no menu do app.
   Requisitos
   a) Criar o model Funcionario.
   b) Criar o service FuncionarioService.
   c) Criar a tela FuncionarioScreen.
   d) Adicionar a opção "Funcionários" ao menu lateral.
   e) Exibir:
      a. Matrícula
      b. Nome
      c. Setor
   f) Não exibir a senha.

2) Exiba um ícone diferente para cada setor
   Manutenção → build
   Produção → factory
   Almoxarifado → inventory

3) Na tela epi_screen.dart faça:
   a. Mostrar a quantidade disponível de cada epi na lista
   b. Destacar em Vermelho os EPIs com CA vencido